

# 棒材组合订单高效锯切优化 —— 技术报告

队伍：鱼不吃猫 参赛编号：AIC-2026-93096493 赛道：AIC-产业命题赛-AI + 钢铁 阶段：复赛 报告日期：2026-09-29

## 摘要

本报告说明我们求解「棒材组合订单高效锯切优化」复赛题的技术路线、关键机制与实验记录。

问题的核心是一个带十二条硬约束的组合优化：把 9,999 个有效订单分配到不超过 2,303 个方案里，每个方案最多 6 轮冷床，每轮受长度（50–150 m）、宽度（2 m）、承重（60 t）三重物理限制，且同一订单若跨轮必须落在相邻轮次上连续锯切。

我们的评分口径由平台回执反向标定得到：**总分 =  $0.4 \times \text{刀子分} + 0.4 \times \text{成材率} + 0.2 \times \text{覆盖率}$** ，其中刀子分 =  $100 \times 160000 / \text{刀数}$ ，无时间项；**官方分等于本地预测分四舍五入到两位小数**。该口径在五个包、十五项子指标上与平台**逐位相同**，且其中最优的一个包是在口径确定**之后**才构造、才提交的，属于一次真正的事前预测而非事后拟合。

方法上我们最终收敛到一条可加性架构：以一张基准计划为底，只在**批次（订单组合）粒度**上做局部替换，因为刀数与用料都是批次上的和、覆盖率恒为满分，所以互不冲突的批次补丁可以**精确相加**，整卷分数是它们的严格函数。这条性质把「搜索更优计划」变成了「并行地、可回滚地寻找单批次改进」，使我们在两天内把本地计划从 92.4322 推到 **92.6085**，官方分从 90.59 推到 **92.58**（另有已打包待提交的 92.6085  $\Rightarrow$  预计 92.61）。

本报告也如实记录三处我们**做错过的推断**、本地校验器的已知缺口，以及未解开的杠杆。

## 一、赛题与约束

### 1.1 输入与输出

输入是两张表：订单表（订单号、钢种、规格/直径、定尺长度、订单重量）与坯料表。输出是一份 JSON，包含若干 **scheme**（订单组合方案），每个 scheme 内含至多 6 个 **round**（冷床轮次），每个 round 给出参与锯切的订单、每个订单分配的**件数与定尺整数倍长度**。

除异常订单外，全部订单必须参加锯切计算。盘面上 10,000 条记录中有 1 条原始重量为负（订单重量 = -25.418 kg），我们将其按题面允许的异常过滤流程**隔离**而非猜测补值，因此有效订单数是 **9,999**。

### 1.2 十二条约束

数据目录下的 `constraints.txt` 给出两类约束。

**工艺设备与计量：**锯切辊道承载棒材最短 50 m、最长 150 m；辊道宽度 2 m；最大承重 60 t；每个订单组合方案使用的冷床轮数不得超过 6 轮。

**基本业务规则：**订单需同钢种、同规格才可组合；同一订单若分在多轮锯切，必须在**相邻轮次**中连续锯切，中间不得插入其他订单、不得跳轮；每轮冷床铸坯总重不低于本轮棒材重；每订单冷床分配总重量不低于该订单重量；订单每次分配长度须为其定尺长度的整数倍；同一订单不得跨方案重复出现；同一方案内订单号不得重复；除异常订单外所有订单均须参加锯切计算。

其中「相邻轮次连续锯切」这一条在实现上最容易出错：它使轮次顺序成为一个有约束的序列而非可自由排序的集合。我们把它编码进计划的合法性判据，并让所有候选在进入评分前必须通过独立校验器。

### 1.3 评分口径

平台的回执给出三个子指标与总分。我们用五张回执（09-26 夜的四发与 09-27 的 `cand_rp4`）做参数识别，得到：

$$\text{总分} = 0.4 \times (100 \times 160000 / \text{刀数}) + 0.4 \times \text{成材率}(\%) + 0.2 \times \text{覆盖率}(\%)$$

权重是这样钉死的：第 1 与第 5 个包只差成材率（-4.18 个点，刀数与覆盖率逐位相同），分差  $1.672 = 0.4 \times 4.18$  —— 若权重是 0.30 则分差只会是 1.254，因此官方 PDF 页内「权重表 30%」与「公式行 40%」的矛盾由此定案为 **40%**。覆盖率权重由第 4 个包相对第 1 个包独立解得 0.200，第 3 个包复算 0.199。刀数权重由  $1 - 0.4 - 0.2$  闭合。

**没有时间项：**五次回分都能在两位小数显示精度内被上式复现，若存在时间项其权重小于 0.005。这意味着运行时长不构成压力，我们因此可以把全部算力投给解质量。

灵敏度（在 18.5 万刀附近）：每 +1,000 刀约 -0.187 分；成材率每 1% 值 0.40 分；覆盖率每 1% 值 0.20 分。裸眼看上去「刀数」与「成材率」是主战场，覆盖率只剩 0.2 分空间，这与我们最终把覆盖率稳定在 100.0000% (9,999/9,999) 后的判断一致。

## 二、求解架构

### 2.1 计划的表示与两级结构

一份计划由**批次**构成：一个批次是一个订单组合（同钢种同规格，可跨多轮连续锯切）。批次内部是一个子问题——给定这批订单，用最少的坯料、最少的刀数把它们锯出来；批次之间则相互独立。

这个两级结构是整套方法的地基。它的价值不在于「分解更好算」，而在于**分解之后改进可以相加**（见第三节）。

### 2.2 求解器与精修器

**求解器**给出可行解：把订单聚成同钢种同规格的批次，为每批决定轮次划分、每轮的订单集合、每单的件数与长度倍数。**精修器 (polish)** 在可行解邻域内做局部下降，反复尝试单轮内的 件数/长度倍数调整、轮次合并等动作，接受任何降低目标值的移动，直到预算耗尽。

精修器是整套流程里调用最频繁的组件，也是所有后续机制的公共底座：链重排之后要走精修，打包之前要走精修，补丁验收时的「等预算对照」也依赖它。

## 2.3 打包环节

打包分两步，这是踩过坑之后固化下来的流程：`src/build_submission.py` 把计划转成提交 JSON 并写出自己 schema 的校验报告，随后 `tools/resync_reports.py` 必须再跑一次，把报告重写成测试与漂移检查期望的 schema。少了第二步，校验报告与下游判据就对不上。

打包环节还承担一个安全职责：它把「本地模型算出来的数」与「平台认证过的数」严格区分开。`resync_reports.py` 恒把 `submission_allowed` 置为 `false` 并附上理由，任何查看候选包的人 **都不应**把 `model_result.platform_check_passed: true` 误读为平台已接受——前者只是本地独立校验器的自查结果。

# 三、关键机制一：批次级可加性

## 3.1 为什么补丁可以精确相加

这是我们在复赛阶段最有价值的一条性质。

设基准计划  $P_0$  由批次  $\{b_1, \dots, b_n\}$  组成，整卷分数为  $S(P_0)$ 。现在对第  $i$  批做一次局部替换，得到新批次  $b_i'$  与补丁计划  $P_i = P_0$  除了  $b_i$  换成  $b_i'$ 。

在评分公式下：刀数是各批次刀数之和；用料 (bill) 是各批次用料之和；覆盖率是全局项，且在我们的盘面上恒为 9,999/9,999。

因此当两个补丁  $P_i$ 、 $P_j$  ( $i \neq j$ ) 都合法时，把两个替换同时应用得到  $P_{\{ij\}}$ ，其刀数与用料 **恰为** 两个补丁各自增量的和，覆盖率不变——于是  $S(P_{\{ij\}})$  可被精确算出，既不需要重跑求解器，也不存在「两个改进互相抵消」的风险。

这条性质把优化问题从「在巨空间里找一个好计划」改写成「在  $n$  个互不干扰的小空间里各自找改进」，后者天生可并行、可增量、可回滚。

## 3.2 合并算法

合并器 `runs/merge_reseed.py` 按如下规则工作：

1. 对每个补丁，核对它相对基准 **只动了它自己声明的那一批**；
2. 批次索引互不相同的补丁，直接整批应用（无争议）；
3. 若两个或更多补丁落在 **同一批**上（不同轮次各自赢下了这一批），这些批次进入「争议集」，

由整卷分数**逐个贪心**结算：把候选按顺序试放进当前最好的计划里，只保留真的提高整卷分的那个；

1. 若最终结果不优于基准，整个合并回退。

过程中每接受一个候选就打印一次整卷分的增量，因此合并日志本身就是一份可审计的证据（本次 plan\_r14 的合并日志里可以逐个看到每一批带来的 +0.0001...+0.0007）。

### 3.3 安全守卫

合并有一个硬守卫：`changed(base, patch) == [bi]`。任何**越界修改**（补丁动了别的批次）或**残缺文件**（扫描还在写它）都会被直接跳过而不是应用。

这不是洁癖。`runs/` 目录里同时存在十几条扫描前缀的产物，文件名是 `runs/<prefix>_patch_<bi>.json`，一旦某个补丁因为建表缺陷写错了批次号，没有这条守卫就会静默污染整卷计划。我们确实遇到过一次列表缺陷（拼表时源文件缺尾换行，导致两个批号粘连成一个非法索引），守卫把它的影响限制在「少探一个批」，没有扩散成错误计划。

## 四、关键机制二：链探针不是种子穷尽

### 4.1 链重排与它的种子依赖

「链」指的是方案内各轮次的顺序与衔接方式。同一批订单、同一组轮次，**换一种排列方式** 会改变每轮的宽度利用与刀数，因此存在「重排更优」的空间。

链探针的做法是：给定一批订单，用扰动后的链重新铺设，再走同一份精修预算，然后比较。我们把三条路线区分开：

- **A0** = 计划原本的批次 + 一次生产用精修；
- **A1** = 同一订单序列由重排器重新铺设 + 同一份精修；
- **A2** = 扰动链 + 同一份精修。

**route B 的验收判据**是  $A2 < A0$  且  $A2 < A1$  且形状确实改变。要求同时压过 A1 是为了排除「改进其实来自精修多跑了几步」这种假信号。`--out` 只在真的验收通过时落盘，所以 `runs/<prefix>_patch_*.json` 的**文件个数就是胜出批数**，不需要另建账本。

### 4.2 route A 的关闭（一条被自己证伪的推断）

我们起初想把 A1 本身当作候选：`chain_probe --out-relay` 对 A1 的整卷状态评分，只在改善时落盘。基于「普查阶段已经观察到哪些批重排有益」这一前提，我们曾推断「全扫 route A 没有价值，因为普查已把它筛完」。

**这条推断是错的。** 用 bi=1017 做对照实验：种子 20260928 下 A1 复现普查原值 555,738.2，换成种子 20260929 后变成 563,872.7（而 A0 两次逐位相同）。原因是 A1 重排之后仍要走精修，而**精修吃种子**——所以 A1 是随种子变的，「普查已经筛完 A1」这个前提不成立。

修正后的判据不是推断而是实测：用 60 批随机试点全扫 route A，**0 胜 / 60**（36 批重排真的动了，但进入整卷评分的 15 条里 13 条为负）。再把 route A 在另一批上的 9 个落选批次换种子重跑，**仍 0 胜**。于是关闭 route A：它的收益全部集中在「普查已观察到重排有益」的那一小撮批上，离开这个集合收益为零。route A 最终只贡献 12 条补丁。

### 4.3 列表划分

普查阶段用固定种子 20260928 扫过 1,703 批，多种子重探又覆盖了 457 批，两者**互不重叠**，差集是**1,245 批从未被第二个种子碰过的批次**。我们的换种子扫描因此分两条列表：chain\_fresh\_list.txt（1,245 批）与 reseed\_list.txt（457 批）。

这个划分带来一个可检验的预测：未重探区的产出密度应当更低。实测确实如此——1,245 批上的前几轮胜率 1.4% / 1.2% / 0.96%，而反复采过的 457 批在同轮出 2.0%–3.3%。这与此前记录的普查自身衰减曲线（1–60 名 53%、61–260 名 38%、261–560 名 23%、561–1707 名 8%）方向一致。我们此前「整片新地能翻两倍产出」的乐观外推**没有成立**。

## 五、实验结果

### 5.1 官方分数轨迹

累计 15 次真实提交，全部被平台受理为**可行**，无一次结构性失败。官方分轨迹：

| 时间                     | 官方分                           | 备注                                      |
|------------------------|-------------------------------|-----------------------------------------|
| 2026-09-26 20:22–20:37 | 90.59 / 88.92 / 89.87 / 90.40 | 用于识别评分公式的四发                             |
| 2026-09-27 07:06       | <b>92.12</b>                  | cand_rp4；事前预测 92.1180，事后逐位命中            |
| 2026-09-27 22:31–23:26 | 92.16 / 92.43 / 92.43 / 92.48 | 当日五跳                                    |
| 2026-09-28             | 92.52 / 92.54 / 92.56 / 92.57 | 早前四发                                    |
| 2026-09-28 23:01:52    | <b>92.58</b>                  | 当前官方最高（刀 184,444 / 材 94.69% / 覆 100.0%） |

### 5.2 本地计划轨迹

| 计划           | 本地分     | 相对基准                             |
|--------------|---------|----------------------------------|
| plan_r6（基准底） | 92.5831 | —                                |
| plan_r9      | 92.5935 | +0.0104                          |
| plan_r10     | 92.5982 | +0.0151（首次跨过 92.5951 ⇒ 官方 92.60） |

| 计划       | 本地分            | 相对基准                                   |
|----------|----------------|----------------------------------------|
| plan_r11 | 92.6028        | +0.0197                                |
| plan_r12 | 92.6034        | +0.0203                                |
| plan_r13 | 92.6047        | +0.0216                                |
| plan_r14 | <b>92.6085</b> | <b>+0.0254 (跨过 92.6051 ⇒ 官方 92.61)</b> |

plan\_r14 的报告：刀 **184,415**、成材率 **94.7603%**、覆盖率 **100.0000% (9,999/9,999)**、方案 2,303 个、轮次 10,233。独立校验器返回 `passed: true`、0 违规。

四舍五入线的意义：**92.6051 是「官方记 92.61」的门槛**。plan\_r14 的 92.6085 已在其上，`artifacts/candidates/cand_r14/` 已按流程打包完成（ZIP 127,493 B, sha256 `03c1faff...`，内含单一同名 JSON），并已通过只读预检。

### 5.3 换种子扫描的产出

十轮换种子扫描共同构成 plan\_r14 的主要增量来源（route A 只贡献 12 条）。单轮整卷收益区间为 **+0.0006...+0.0047**，胜出批数从 12 到 35 不等。

这里有一个必须写清楚的观察：**胜出数是噪声主导的**。同一 1,245 批上的四个种子依次出 18 / 15 / 12 / 24 条；457 批上的两轮出 6 / 15 条。第三轮和第七轮都只给了 +0.0006，而第五轮给了 +0.0047 —— 相差近八倍，且轮次之间没有可辨的衰减规律。

### 5.4 分数模型的校验强度

第 1.3 节的公式不是「拟合得很像」，而是**逐位相同**：五包 × 刀/材/覆 = 15 项子指标 全部数字相同，五个总分取整后全等。且差值不是常数（+0.0020 / -0.0020 / 0.0000），这排除了「官方分 = 本地分 + 常数」的伪模型，指向正确的「四舍五入」。

一个我们主动声明的**残留不确定性**：本地未取整分四舍五入，与「官方先各自取整到两位再加权」，这两种算法在五包上给出同一个两位小数，但原理上会在真值距 `.005` 边界极近时差 0.01。当前无法区分，因为五包的真值离最近边界都有 0.0010 以上。我们把这个风险写进文档而不是忽略它——它正是「本地通过与平台认证是两件事」的一个具体实例。

## 六、工程实践中的四个坑

这一节记录与算法无关、但每次都会浪费数小时的工程问题，供复现者参考。

**其一：运行中的脚本不能改。** bash 是边读边解析边执行的，读完已执行的部分后会回到文件里续读。一次扫描运行中途给驱动脚本加了一个参数，导致 bash 按旧偏移量读到了新内容，在 `xargs` 那行报语法错误并 `exit 2`。**后果是「看起来失败、其实成功」**：1,245 个工作项全部跑完、15 个胜出件齐全，只有脚本末尾的收尾回显没执行。正确处置是**先按产物清点证据**（日志条数、产物个数、抽查内容），不要急着判失败或重跑。

**其二：中止一条并行扫描要杀父进程树。** 该扫描由 `xargs -P N` 启动，进程树形如 `claude.exe → bash → bash → bash → python`。TaskStop 与 MSYS 的 `pkill` 都杀不掉 Windows 侧的 `python.exe` 子进程：前者报成功但链还活着，后者杀掉后 `xargs` 立刻补位重开。更隐蔽的是判据错误——用 `grep -c chain_probe` 数进程会把命令行里恰好含该字样的 `bash` 包装层算进去，数目虚高。正确判据是 `Win32_Process` 里 `Name -eq 'python.exe'` 且命令行含脚本名，然后按**从根到叶**的顺序 `Stop-Process`。收手后要用「叶子进程数归零 + 产物数不再增长」两条确认。

**其三：这台机器上「提交量」比「空闲内存」更能预测卡死。** 本机有内存耗尽导致周期卡死的历史。一次扫描运行中，空闲物理内存落到 272–475 MB（看着只是紧张），但**系统提交量已达 31,649 / 32,464 ≈ 97.5%**——这才是「下一次分配失败就会冻机」的直接信号。空闲内存会被 `standby cache` 吸收而上下抖动，会骗人。当时我们的整条链只占 453 MB，真正的消耗者是模拟器（2.7 GB）与容器（813 MB）；但在只剩约 800 MB 提交余量时，我这一份确实是压垮的那一点，所以仍然收手。

**其四：策略层封禁会静默把整批扫描变成假产物。** 一次重跑中 1,027 条日志全部只有 60 字节，内容是 `Permission denied`——系统的应用控制策略（Device Guard / WDAC）拦住了虚拟环境里的 `python` 解释器。**危险之处在于日志文件确实生成了**：如果只按「日志条数」判断进度，会误以为这一轮跑过了。处置是改用真正的解释器，并**先删除这批假产物**再重启，否则后续的计数判据全部失效。

## 七、诚实边界

### 7.1 三次被打脸的「量级推断」

我们在文档里写过三次「下一轮就够了」式的乐观推断，三次都被实测打脸：

1. 第一次推算「再跑两轮种子就能到 92.6051」——实际五、六轮的产出低于该外推；
2. 第二次写「再一轮种子大概率跨过」——下一轮只给了 +0.0006；
3. 第三次（`plan_r13` 落地时）缺口只剩 +0.0004，「大概率一轮就够」——

实际下一轮给了 +0.0038，这次成立，但**成立与否并不由推断决定**。

结论：**换种子扫描的收益是噪声主导的，任何「再一轮就够了」的推断都不成立**；只能说「期望约 +0.002/轮，需要跑几轮」。我们把这三条写进文档而不是悄悄删掉，因为它们正是复现者最容易重复犯的错。

### 7.2 本地校验器的已知缺口

**本地校验通过不等于平台接受**，这是本项目的核心风险认知：

- 本地校验器是对约束的**再实现**，不是平台评分器本身；
- 「相邻轮次连续锯切」的判据来自平台 PDF 的四种可能读法，我们通过四次官方回执（零违规/可行）把其中一种**认证**为正确，但这仍是对语义的推断而非对源码的读取；
- 打包流程恒把 `submission_allowed` 置为 `false` 并在报告里写明理由，

就是为了防止下游把本地自查误读为平台认证；

- 「官方分 = 本地分四舍五入」在 `.005` 边界附近的两种口径无法区分（见 5.4）。

## 7.3 未解开的杠杆

以下方向我们**确认过无效或未穷尽**，如实列出：

- **route A（把平铺重排本身当候选）**：已关闭，见 4.2；
- **交叉算子（链与窗口互为不动点）**：一次负结果——链与窗口互相引向对方的不动点，

不存在可反复套利的「乒乓」杠杆；

- **窗口宽度**：`window=3` 是一条被锁死的预算下限，全量扫描后确认无剩余空间；
- **超产削减**：被 48 m 的床面长度下限锁死，超产既不扣分也不计入成材率分子，故不动它；
- **求解器成材率分子口径**：两种口径对合法计划塌成常数，差别只是一个约 1% 的边际价格偏差，

等预算对照给出  $\Delta S = -0.0143$ ，因此不换。

剩余的可行方向只有一条：**继续换种子扫描**。它是可靠但收益递减的，单轮期望约  $+0.002$ ，而下一档「官方 92.62」需要本地  $\geq 92.6151$ ，即还需要  $+0.0066$ 。

# 八、可复现性

## 8.1 目录约定

源码在 `src/`，测试在 `tests/`，产物在 `artifacts/runs/`（计划）与 `artifacts/candidates/`（候选包），扫描中间件在 `runs/`（约 41 MB，不入版本库）。命令统一走 `aic.py`。

## 8.2 主要复现命令

```
# 校验一份提交 JSON（复赛口径）
python -X utf8 aic.py check artifacts/runs/requant/plan_r14.json --round semi --data data/semi

# 打包成候选（两步，缺一不可）
python -X utf8 src/build_submission.py --input artifacts/runs/requant/plan_r14.json \
    --team 鱼不吃猫 --output-dir artifacts/candidates/cand_r14 --round semi --data data/semi
python -X utf8 tools/resync_reports.py artifacts/candidates/cand_r14 --data data/semi

# 一份批次索引列表上的换种子链扫描（--out 只在验收通过时落盘）
bash runs/_chain_reseed.sh artifacts/runs/requant/plan_r6.json <seed> <prefix> \
    runs/chain_fresh_list.txt 5
```

```
# 多路合并（各路线必须同底，才能精确相加）
RESEED_BASE=artifacts/runs/requant/plan_r6.json \
RESEED_PREFIXES=r12,r14,rsg,rsh,rsi,rsj,rsk,rsl,rsm,rsn,rso \
RESEED_OUT=artifacts/runs/requant/plan_r15.json \
python -u -X utf8 runs/merge_reseed.py
```

本机注意：虚拟环境里的解释器被应用控制策略拦截，上述命令需用真实解释器执行；生成 PDF 报告用 `tools/report_md_to_pdf.py`（Markdown → HTML → 浏览器 headless 打印）：

```
python -X utf8 tools/report_md_to_pdf.py docs/REPORT_TECH_20260929.md \
--out docs/REPORT_TECH_20260929.pdf
```

本报告即由该命令生成（A4、9 页、内嵌中文字体），源文件是本 `.md`，PDF 是产物。

## 8.3 证据

- 评分公式的标定与五包逐位对账：`docs/SCORE_MODEL.md`
- 五张回执原始行（含 `attachmentSha256` 与匹配标记）：`artifacts/accepted/official_scores.json`
- 逐日状态与每次实验的结论：`docs/STATUS_20260928.md`、`docs/CURRENT.md`
- 候选包与校验报告：`artifacts/candidates/`

## 附录 A：三个统计口径的细节

**刀数的取整口径。** 每轮刀数等于该轮总段数加一（连接处各一刀、每轮冷床一组头尾），不乘棒材根数。件数按**四舍五入**（snap）而非向下取整：17,029 个书写条目里有 8,003 条在浮点下是整倍数少一个 ulp，向下取整会差一把。平台的实报值与本地 snap 值逐位一致。

**成材率的分子。** 平台读的是  $\sum \min(\text{实际交付}, \text{需求})$ 。我们曾误以为「官方成材率 = 本地 × 1.0104」，四发比值一致，但那是**本地旧代码的口径偏差**——旧代码把分子算成「直径截断到整毫米」的初赛口径质量，偏小约 1%。修正后同一份包的本地值直接等于官方值。**这不是把模型拟合到答案上**：修正在最优包构造与提交之前完成，它的三项子指标在提交前就由本地投影写下、提交后与平台读数逐位相同。

**覆盖率的分母。** 覆盖率分子只数「与别人同处一轮」的订单，因此一个被孤立的订单会静默造成  $0.2 \times 100 / 9999 \approx 0.002$  分的损失。我们把它按材料当量定价（约 28,838 kg-eq）纳入优化目标。

## 附录 B：术语

- **批次 (batch)**：一个同钢种同规格的订单组合，是补丁与合并的最小单位。
- **scheme / round**：输出 JSON 里的方案 / 冷床轮次。

- **polish (精修)**：在可行解邻域内做局部下降的精修器，吃随机种子。
- **A0 / A1 / A2**：原批次 / 同序列重排后 / 扰动链，三者的等预算对照见 4.1。
- **route A / route B**：两种链相关招式；route A 已关闭，route B 是 `plan_r14` 的主要增量来源。
- **四舍五入线**：使官方分跨过下一个 0.01 的本地分门槛（如  $92.6051 \Rightarrow 92.61$ ）。